Sentinel AI Security Audit Report

****Audit Metadata****

- **Filename**: test_login.c
- **SHA-256 Checksum**: 8ee428828bed104589656fe3fbac83f7d5ec8526c102eaef5dbe5101150c2592
- **Verdict**: **VULNERABLE**
- **Risk Level**: **CRITICAL**
- **Total Functions Evaluated**: 2

****Executive Summary****

The Sentinel AI security audit has concluded its assessment of `test_login.c`. Our proprietary GNN-based analysis engine, utilizing the GATv2 (Graph Attention Network) architecture, has identified a **CRITICAL** memory safety vulnerability within the codebase.

The audit reveals a high-risk exploit vector in the login handling logic. The identified vulnerability allows for an out-of-bounds write, which can be weaponized to achieve stack-based buffer overflows. In a production environment, this flaw could lead to unauthorized administrative access, arbitrary code execution (ACE), or a complete system crash (DoS). Immediate remediation is required before this code is integrated into the production branch.

****Analysis Metrics****

- **Static Analysis Engine**: GNN-Compiler-Wrapper (GATv2 Model)
- **Total Functions Scanned**: 2
- **Flagged Functions**: 1
- **Model Confidence Score**: 70.02%
- **Analysis Depth**: Control Flow Graph (CFG) & Instruction-Level Heuristics

---

****Detailed Findings****

Finding 1: Function `vulnerable_login`

- **Model Confidence**: 70.02%
- **Identified Threat**: CWE-119 (Memory Safety / Buffer Overflow) / CWE-787 (Out-of-bounds Write)
- **Rule Violation**: SEI CERT C Rule ARR30-C ("Do not form or use out-of-bounds pointers or array subscripts")

Technical Breakdown of Exploit Vector:

The disassembled CFG reveals a classic stack-based buffer overflow. In **Basic Block 0**, the function allocates `0x30` (48 bytes) for the stack frame (`SUB RSP, 0x30`). However, the local buffer used for the input operation is located at `[RBP - 0x10]` (16 bytes from the base pointer).

The instruction `CALL 0x00003c70` invokes an input-handling function (likely `gets` or an unbounded `strcpy`). Because the destination buffer at `[RBP - 0x10]` only has 16 bytes of allocated space before it begins overwriting the saved Frame Pointer (RBP) and the Return Address, any input exceeding this length will grant an attacker control over the instruction pointer (RIP). This is a textbook case of an out-of-bounds write where the software fails to validate the length of the input relative to the destination buffer size.

Remediation & Secure Code Fix:

To mitigate this vulnerability, the developer must replace unbounded memory copy/input functions with bounds-checked alternatives. The use of `fgets` or `strncpy` is recommended to ensure that the input size never exceeds the allocated buffer capacity.

Vulnerable Pattern (Conceptual):

```
void vulnerable_login(char *input) {
    char buffer[16];
    strcpy(buffer, input); // CRITICAL: No bounds checking
    // ... logic ...
}
```

Secure Remediation (Recommended):

```
#include <string.h>
#include <stdio.h>

#define MAX_BUF 16

void secure_login(const char *input) {
    char buffer[MAX_BUF];

    // Use strncpy to limit copy size and manually ensure null-termination
    strncpy(buffer, input, MAX_BUF - 1);
    buffer[MAX_BUF - 1] = '\0';

    // Alternatively, if reading from stdin:
    // fgets(buffer, sizeof(buffer), stdin);

    // Proceed with secure comparison
    if (strncmp(buffer, "SECRET_PASS", MAX_BUF) == 0) {
        printf("Access Granted\n");
    } else {
        printf("Access Denied\n");
    }
}
```

General Mitigations & Best Practices

To harden the application against memory corruption exploits, the following system-level protections and development practices should be implemented:

1. **Stack Canaries (Stack Smashing Protector)**: Enable compiler flags (e.g., `-fstack-protector-all`) to insert "canary" values before the return address. If a buffer overflow occurs, the canary is corrupted, and the program terminates before the attacker can hijack control flow.
2. **Address Space Layout Randomization (ASLR)**: Ensure the target environment has ASLR enabled to randomize the memory addresses of the stack, heap, and libraries, making it significantly harder for attackers to predict jump targets.

3. **Data Execution Prevention (DEP/NX)**: Mark the stack as non-executable to prevent the execution of shellcode injected into buffers.
4. **Control Flow Integrity (CFI)**: Utilize modern compiler features that validate the integrity of the program's control flow graph at runtime, preventing unauthorized jumps to arbitrary code.
5. **Static & Dynamic Analysis (SAST/DAST)**: Integrate automated security scanning into the CI/CD pipeline to catch memory safety violations during the development phase.

End of Report

Sentinel AI Security Division